

Background Subtraction

1 Code Explanation

1.1 Design Choices

For the final output I put the bounding boxes around the original frames. It looks nicer to see the output in color instead of a gray-scale video.

1.2 External Resources Used

1.2.1 VS Code and Video Player

I had some issues with VS Code displaying the frames in the video for a while. I used Gemini to try and help me debug what was going on, but nothing worked. After a while I gave up and loaded the `.ipynb` into a Google Colab environment and that worked just fine. I think I am going to use that from now on.

1.2.2 Scikit-image Documentation

For the implementation of showing the `'gray'` and `'viridis'` color schemes, I looked at the Scikit-image documentation. I also used that documentation to look at the `threshold_otsu` implementation.

2 Discussion Questions

1. When you compute the average image (background image), the cars have disappeared – why?

The parts that are moving (changing pixel values) are drowned out by the background that is constant. If a portion of the road is always black except for the two frames that a red car passes through it, the average of all the frames would still be relatively black.

2. How well does the technique work to separate the cars from the background? Where does it fail and why?

In my implementation there are individual white blobs that show up with small bounding boxes around them. I think this is happening because the threshold separates these

blobs from the actual “object”. This leads to small bounding boxes within a larger moving object.

Another issue that I can foresee happening is false car identification. This algorithm works for anything moving, not just a car. If a bird or animal were to cross the road, then that would mess up the averaged background image and result in a bounding box around that.